

# Containers

Laboratórios de Sistemas e Serviços

---

Mário Antunes

Universidade de Aveiro

2026

# Introdução

---

# O Que São Contentores?

Um **contentor** é uma unidade de software padrão e executável que empacota o código de uma aplicação juntamente com todas as suas dependências de tempo de execução — bibliotecas, ficheiros de configuração e ferramentas do sistema.

Este pacote é **isolado**, garantindo que a aplicação corre de forma uniforme e consistente em qualquer anfitrião compatível.

**Analogia:** Um contentor é como um contentor de transporte padronizado. Não importa o que está lá dentro; pode ser manuseado por qualquer navio, camião ou grua compatível (máquina anfitriã).

# Terminologia

Antes de começarmos, vamos definir alguns termos-chave.

- **Imagem (Image):** Um modelo inerte, apenas de leitura, que contém uma aplicação e as suas dependências. Pense nisto como uma **planta** ou uma classe em programação orientada a objetos.
- **Contentor / Instância (Container / Instance):** Uma **instância** executável de uma imagem. Esta é a aplicação real, a correr (como um objeto criado a partir de uma classe).
- **Registo (Registry):** Um sistema de armazenamento para imagens de contentores. O **Docker Hub** é um registo público popular.
- **Motor / Runtime (Engine / Runtime):** O software que constrói, executa e gere contentores (p. ex., Docker Engine, Podman).
- **Volume:** Um mecanismo para persistir dados fora do sistema de ficheiros efêmero de um contentor.

# O Problema: “Na Minha Máquina Funciona!”

Todos os programadores já enfrentaram este problema clássico:

- A sua aplicação funciona perfeitamente no seu portátil (que tem Python 3.9, uma versão específica de uma biblioteca, e corre Debian).
- Quando a entrega a um colega (que tem Python 3.8 e corre macOS) ou a implementa num servidor (a correr um SO mais antigo), ela falha.

Estas diferenças nos ambientes criam um enorme desafio para a portabilidade e reprodutibilidade do software.

# A Solução: Contenedores

Os contentores resolvem este problema empacotando **tudo o que a aplicação precisa** num único pacote autónomo.

- O código da aplicação.
- O runtime da linguagem (p. ex., Python 3.9, Node.js 20).
- Todas as bibliotecas necessárias e as suas versões exatas.
- Dependências ao nível do sistema e configuração.

O contentor corre de forma idêntica no portátil do programador, num servidor de CI/CD ou numa instância cloud de produção.

# Fundamentos de Contentores

---

# Como o Isolamento é Alcançado: Namespaces

Os contentores correm à **velocidade máxima do hardware** porque são apenas processos isolados no kernel do anfitrião. O isolamento é fornecido pelos **Namespaces do Linux**.

Os Namespaces virtualizam os recursos do sistema para um processo, fazendo parecer que este tem a sua própria cópia privada. Os namespaces-chave incluem:

- **PID:** Isola os IDs dos processos. Dentro do contentor, a aplicação é o PID 1.
- **NET:** Fornece uma pilha de rede isolada (endereços IP, tabelas de encaminhamento).
- **MNT:** Isola os pontos de montagem do sistema de ficheiros.
- **UTS:** Isola o hostname e o nome de domínio.
- **USER:** Mapeia UIDs/GIDs do contentor para UIDs/GIDs diferentes no anfitrião.



# Como os Recursos são Geridos: Cgroups

Para evitar que um contentor consuma todos os recursos do sistema, o kernel do Linux usa **Control Groups (cgroups)**.

Os Cgroups permitem que o anfitrião limite e monitorize os recursos que um contentor pode usar:

- Uso de CPU (p. ex., limitar a 1 núcleo de CPU).
- Memória (p. ex., limitar a 512 MB de RAM).
- Largura de banda de I/O de disco.
- Largura de banda de rede.

**Analogia:** Os Namespaces são as **paredes** entre apartamentos. Os Cgroups são os **contadores de serviços públicos e disjuntores**, garantindo que nenhum inquilino pode usar todos os recursos do prédio.

- **Máquinas Virtuais (VMs)** virtualizam o **hardware**. Cada VM inclui uma cópia completa de um SO convidado e do seu kernel. São pesadas e demoram minutos a arrancar.
- **Contentores** virtualizam o **sistema operativo**. Partilham o kernel do sistema anfitrião e são leves, arrancando em segundos.

# VMs vs. Contentores: Comparação

| Característica           | Máquinas Virtuais                   | Contentores                                    |
|--------------------------|-------------------------------------|------------------------------------------------|
| <b>Analogia</b>          | <b>Casas:</b> Totalmente autónomas. | <b>Apartamentos:</b> Partilham infraestrutura. |
| <b>Abstração</b>         | Virtualização de Hardware           | Virtualização de SO                            |
| <b>Tamanho</b>           | Gigabytes (GB)                      | Megabytes (MB)                                 |
| <b>Tempo de Arranque</b> | Minutos                             | Segundos ou menos                              |
| <b>Desempenho</b>        | Sobrecarga baixa a média            | Muito baixa (quase nativa)                     |
| <b>Uso de Recursos</b>   | Mais elevado (SO completo por VM)   | Mais baixo (kernel partilhado)                 |
| <b>Isolamento</b>        | <b>Forte</b> (Nível de hardware)    | <b>Bom</b> (Nível de processo)                 |
| <b>Portabilidade</b>     | Portátil (mas grande)               | <b>Extremamente Portátil</b>                   |

# A Imagem do Contentor e as Suas Camadas

Uma **imagem** é um modelo apenas de leitura construído a partir de uma série de **camadas** empilhadas. Cada instrução num `Dockerfile` cria uma nova camada.

- **Camada base:** A imagem de SO inicial (p. ex., `alpine`, `ubuntu`).
- **Camadas intermédias:** Cada instrução `RUN`, `COPY` ou `ADD` adiciona uma camada.
- **Camada de topo (camada do contentor):** Uma camada fina e gravável adicionada quando um contentor arranca.

Isto torna os builds rápidos e o uso de disco eficiente, já que múltiplas imagens podem partilhar camadas base comuns.

# Dados Persistentes: Volumes

Por defeito, o sistema de ficheiros de um contentor é **efémero** — é apagado quando o contentor é removido.

Para guardar dados permanentemente, usam-se **volumes**:

- **Volumes nomeados:** Geridos pelo Docker/Podman. Ideais para bases de dados e dados persistentes de aplicação. Exemplo: `docker volume create mydata`.
- **Bind mounts:** Mapeiam um diretório específico do anfitrião para dentro do contentor. Ideais para desenvolvimento, onde se querem alterações de código em tempo real. Exemplo: `-v ./src:/app/src`.

**Regra fundamental:** Nunca armazene dados importantes apenas na camada gravável de um contentor.

O motor de contentores cria uma **rede virtual em modo ponte (bridge)**. Os contentores na mesma rede recebem um IP privado e podem comunicar entre si.

- **Mapeamento de Portas:** Para expor o serviço de um contentor ao mundo exterior, mapeia-se uma porta do anfitrião para uma porta do contentor (p. ex., -p 8080:80).
- **DNS Interno:** Ao usar o Docker Compose, cada serviço pode alcançar outro usando o nome do serviço como hostname. O código da sua webapp pode simplesmente conectar-se a `http://database:5432` para chegar ao contentor da base de dados.

A **Open Container Initiative (OCI)** define normas da indústria para formatos e runtimes de contentores.

- **Especificação de Imagem:** Define o formato das imagens de contentores, garantindo que qualquer imagem compatível com OCI funciona com qualquer runtime compatível com OCI.
- **Especificação de Runtime:** Define como executar um contentor (a implementação de referência é o `runc`).
- **Especificação de Distribuição:** Define como enviar/descarregar imagens de/para registos.

Como Docker, Podman e outras ferramentas seguem todas as normas OCI, as imagens construídas com uma ferramenta funcionam perfeitamente com outra.

# Docker

---



# Apresentando o Docker

O Docker é a plataforma que popularizou os contentores. Fornece um conjunto simples de ferramentas para construir, distribuir e executar qualquer aplicação, em qualquer lugar.

- **Docker Engine:** O serviço de fundo (daemon) que gere os contentores.
- **Docker CLI:** A ferramenta de linha de comandos que usa para interagir com o Docker Engine.
- **Docker Hub:** Um registo público de imagens de contentores pré-construídas.
- **Docker Desktop:** Uma aplicação com GUI para Windows e macOS que inclui o Engine, CLI e Compose.

O Docker usa um modelo **cliente-servidor**:

1. O **Docker CLI** (cliente) envia comandos para o **Docker Daemon** (dockerd).
2. O daemon faz o trabalho pesado: construir imagens, executar contentores, gerir redes e volumes.
3. O daemon descarrega imagens de um **Registo** (p. ex., Docker Hub) quando necessário.

O daemon corre como root e escuta num socket Unix. Esta é uma diferença arquitetural chave em relação ao Podman.

## Comandos Docker Comuns

| Comando                                  | Descrição                                                      |
|------------------------------------------|----------------------------------------------------------------|
| <code>docker run [imagem]</code>         | Cria e inicia um novo contentor a partir de uma imagem.        |
| <code>docker ps</code>                   | Lista contentores em execução. <code>ps -a</code> lista todos. |
| <code>docker stop [id/nome]</code>       | Para um contentor em execução de forma controlada.             |
| <code>docker rm [id/nome]</code>         | Remove um contentor parado.                                    |
| <code>docker logs [id/nome]</code>       | Obtém os logs (stdout/stderr) de um contentor.                 |
| <code>docker exec -it [id]<br/>sh</code> | Abre uma shell interativa dentro de um contentor em execução.  |

# O Dockerfile: Instruções Principais

Um Dockerfile é uma receita para construir uma imagem de contentor. Aqui estão as instruções mais comuns:

- FROM: Especifica a imagem base sobre a qual construir (p. ex., `ubuntu:22.04`).
- WORKDIR: Define o diretório de trabalho para os comandos seguintes.
- COPY: Copia ficheiros ou diretórios do anfitrião para a imagem.
- RUN: Executa um comando durante o build (p. ex., `RUN apt-get install -y nginx`).

- CMD: Fornece o comando padrão a executar quando um contentor arranca.
- ENTRYPOINT: Configura o contentor para ser executado como um executável.
- EXPOSE: Documenta em que portas o contentor escuta em tempo de execução.
- ENV: Define variáveis de ambiente persistentes.
- ARG: Define variáveis de tempo de build (não disponíveis em tempo de execução).

## Exemplo de Dockerfile: Um Serviço de Logs

Este Dockerfile simples cria um serviço cujo único trabalho é imprimir um carimbo de data/hora a cada 5 segundos. É útil para testar o comando `docker logs`.

```
# Usar uma imagem base mínima
```

```
FROM alpine:latest
```

```
# O comando a executar quando o contentor arranca.
```

```
# Um ciclo infinito que imprime a data e espera.
```

```
CMD ["sh", "-c", \  
    "while true; do \  
        echo \"[LOG] Servidor a correr em $(date)\"; \  
        sleep 5; \  
    done"]
```

Para construir e executar:

# Construir a imagem e dar-lhe uma tag

```
$ docker build -t logging-service .
```

# Executar o contentor em modo detached

```
$ docker run -d --name logger logging-service
```

# Seguir os logs em tempo real

```
$ docker logs -f logger
```

Escrever Dockerfiles eficientes torna as imagens **mais pequenas, mais rápidas de construir e mais seguras**.

- **Usar imagens base pequenas:** Preferir variantes alpine ou -slim em vez de imagens completas ubuntu/debian.
- **Combinar comandos RUN:** Cada RUN cria uma camada. Encadear comandos com && para reduzir camadas.
- **Ordenar instruções por frequência de alteração:** Colocar instruções que mudam raramente (p. ex., apt install) antes das que mudam frequentemente (p. ex., COPY . .) para maximizar hits no cache de camadas.
- **Usar .dockerignore:** Excluir ficheiros como .git/, node\_modules/ e artefactos de build do contexto de build.



# Builds Multi-Estágio

Os builds multi-estágio permitem usar múltiplas instruções FROM num Dockerfile para **separar ferramentas de build da imagem final de runtime**.

# Estágio 1: Construir a aplicação

**FROM** golang:1.22 **AS** builder

**WORKDIR** /app

**COPY** . .

**RUN** go build -o myapp .

# Estágio 2: Criar uma imagem de runtime mínima

**FROM** alpine:latest

**COPY** --from=builder /app/myapp /usr/local/bin/

**CMD** ["myapp"]

A imagem final contém **apenas o binário compilado**, não toda a toolchain Go. Isto pode reduzir o tamanho da imagem de centenas de MB para apenas alguns MB.

# Docker Compose

---

# Docker Compose: Visão Geral

O **Docker Compose** é uma ferramenta para definir e gerir **aplicações multi-contentor** usando um único ficheiro YAML.

Em vez de executar múltiplos comandos `docker run` com flags complexas, descreve-se toda a pilha aplicacional num ficheiro `compose.yml` e gere-se com comandos simples:

```
$ docker compose up -d           # Iniciar todos os serv
$ docker compose down           # Parar e remover todo
$ docker compose logs -f        # Seguir logs de todos
$ docker compose ps             # Listar serviços em e
```

Um ficheiro `compose.yml` usa estas chaves comuns:

- `services`: A chave raiz onde todos os serviços da aplicação são definidos.
- `image`: Especifica uma imagem pré-construída de um registo.
- `build`: Especifica o caminho para um `Dockerfile` para construir a imagem do serviço.

- `ports`: Mapeia portas do anfitrião para o contentor (p. ex., `"8080:80"`).
- `volumes`: Monta caminhos do anfitrião ou volumes nomeados no contentor.
- `environment`: Define variáveis de ambiente para o serviço.
- `depends_on`: Define dependências entre serviços, controlando a ordem de arranque.
- `networks`: Atribui o serviço a uma ou mais redes personalizadas.
- `restart`: Define a política de reinício (p. ex., `unless-stopped`, `always`).

# Exemplo Compose 1: Construir uma Imagem NGINX Personalizada

Este exemplo mostra como empacotar os ficheiros do seu site diretamente numa imagem personalizada.

## Estrutura de Ficheiros Necessária:

```
.
├── compose.yml
├── Dockerfile
└── my-website/
    └── index.html
```

## Dockerfile

```
# Usar a imagem oficial do NGINX como base
FROM nginx:alpine
```

```
# Copiar a nossa página web para o diretório raiz d
COPY ./my-website /usr/share/nginx/html
```

## compose.yml

```
services:
  webserver:
    build: .
    ports:
      - "8080:80"
```

## Exemplo 1: Explicação

Neste método, criamos uma **imagem autónoma e portátil** que inclui o código da nossa aplicação.

1. Quando executa `docker compose up`, a diretiva `build: .` diz ao Compose para procurar um `Dockerfile` no diretório atual.
2. O `Dockerfile` começa a partir da imagem base padrão `nginx:alpine`.
3. A instrução `COPY` copia a pasta local `./my-website` para a imagem em `/usr/share/nginx/html`.
4. É criada uma nova imagem personalizada contendo tanto o `NGINX` como a sua página web.
5. Um contentor é iniciado a partir desta nova imagem com a porta 8080 mapeada para a porta 80.



**Conceito-Chave:** A aplicação e o seu código são empacotados juntos. Isto é ideal para **implementações de produção**, já que a imagem resultante é um artefacto consistente e imutável que pode ser executado em qualquer lugar.

## Exemplo Compose 2: Usar um Volume para Servir Conteúdo

Este exemplo usa uma imagem NGINX padrão e injeta conteúdo usando um bind mount.

### Estrutura de Ficheiros Necessária:

```
.  
├── compose.yml  
└── my-website/  
    └── index.html
```

## **compose.yml**

```
services:
  webserver:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./my-website:/usr/share/nginx/html
```

*(Não é necessário Dockerfile para este método)*

## Exemplo 2: Explicação

Este método mantém o seu código na máquina anfitriã e liga-o dinamicamente ao contentor.

1. A diretiva `image: nginx:alpine` vai buscar a imagem padrão do NGINX ao Docker Hub. Nenhuma imagem personalizada é construída.
2. Um contentor é iniciado a partir desta imagem padrão.
3. A diretiva `volumes` cria uma ligação em tempo real entre a pasta `./my-website` no anfitrião e `/usr/share/nginx/html` dentro do contentor.
4. O NGINX lê ficheiros diretamente do disco da sua máquina anfitriã.

**Conceito-Chave:** O contentor não tem estado (stateless) e o código vive no anfitrião. Se alterar o seu ficheiro `index.html`, a alteração é refletida **instantaneamente** sem reconstruir ou reiniciar. Isto é ideal para **desenvolvimento local**.

## Exemplo Compose 3: NGINX com uma Cache Varnish

Este exemplo avançado orquestra dois serviços: um servidor web NGINX e uma cache Varnish que se posiciona à sua frente para acelerar a entrega de conteúdo.

### Estrutura de Ficheiros Necessária:

```
.  
├── compose.yml  
└── varnish/  
    └── default.vcl
```

## **varnish/default.vcl (Configuração do Varnish)**

```
vcl 4.1;
```

```
// Definir o servidor backend de onde o Varnish irá
```

```
// 'nginx' é o nome do serviço no compose.yml.
```

```
backend default {
```

```
    .host = "nginx";
```

```
    .port = "80";
```

```
}
```

## **compose.yml**

### **services:**

# A cache Varnish, exposta ao mundo exterior

#### **cache:**

image: varnish:stable

#### **volumes:**

- ./varnish:/etc/varnish

#### **ports:**

- "8080:80"

#### **depends\_on:**

- nginx

# O servidor web NGINX, apenas interno

#### **nginx:**

image: nginx:alpine

# Sem ports: apenas acessível dentro da rede Do



## Exemplo 3: Explicação

Esta configuração demonstra uma arquitetura multi-camada realista, onde os serviços comunicam internamente.

1. O `compose.yml` define dois serviços: `cache` (Varnish) e `nginx`.
2. Apenas o serviço `cache` expõe uma porta (8080) ao anfitrião. O serviço `nginx` é completamente interno.
3. A configuração do Varnish referencia o hostname `nginx`, que o **DNS interno** do Docker resolve para o IP privado do contentor `nginx`.

## O Fluxo do Pedido:

Browser -> Anfitrião:8080 -> Varnish (Cache) -> NGINX (Origem)

No primeiro pedido, o Varnish vai buscar a página ao nginx e armazena-a em cache. Os pedidos seguintes são servidos diretamente da cache, o que é extremamente rápido.

**Conceito-Chave:** Isto demonstra a **descoberta de serviços (service discovery)** e o padrão de **proxy reverso**, um bloco fundamental na arquitetura web.

# **Ecosystema de Contentores**

---

# A Origem: Linux Containers (LXC)

Antes do Docker, havia o **LXC** (2008).

- O LXC é uma interface de espaço de utilizador para as funcionalidades de contenção do kernel Linux (namespaces e cgroups).
- Fornece um conjunto de ferramentas de mais baixo nível para criar e gerir contentores.
- Os contentores LXC tipicamente executam um sistema `init` completo e são usados para isolar sistemas operativos inteiros — comportam-se mais como VMs leves do que como contentores de aplicação.

O LXC provou que a virtualização ao nível do SO era prática. O Docker pegou nesta base e tornou-a acessível aos programadores de aplicações.

# Docker: O Padrão De Facto

O Docker (2013) pegou na tecnologia subjacente do LXC e construiu um ecossistema de alto nível e amigável ao programador à sua volta.

- Introduziu imagens portáteis e em camadas através do `Dockerfile`.
- Criou um registo centralizado (Docker Hub) para partilhar imagens.
- Focou-se em contentores **centrados na aplicação**: um processo por contentor.
- Esta filosofia tornou-se uma pedra angular da **arquitetura de microsserviços**.

A principal limitação do Docker é a sua **arquitetura baseada em daemon**: o daemon `dockerd` corre como root, o que pode ser uma preocupação de segurança em ambientes multi-tenant ou hardened.

O **Podman** (Pod Manager) é um motor de contentores desenvolvido pela **Red Hat** como um substituto direto e drop-in para o Docker.

- Lançado pela primeira vez em 2018, agora amplamente adotado em distribuições Linux empresariais (RHEL, Fedora, CentOS Stream).
- Totalmente **compatível com OCI**: usa o mesmo formato de imagem e as mesmas normas de runtime que o Docker.
- Incluído por defeito no RHEL 8+ e Fedora, onde o Docker já não é distribuído.

# Arquitetura do Podman: Sem Daemon

A diferença arquitetural mais importante entre o Podman e o Docker é que o Podman **não tem daemon central**.

- **Docker:** CLI -> dockerd (daemon, corre como root) -> containerd -> runc
- **Podman:** CLI -> common (monitor por contentor) -> runc

Cada contentor é um **processo filho direto** do comando Podman (modelo fork/exec). Se o Podman terminar, os contentores continuam a correr sob o common.

Isto elimina o ponto único de falha que o daemon Docker representa.

# Podman: Contentores Sem Root (Rootless)

O Podman foi projetado desde o início para executar contentores **sem privilégios de root**.

- Os contentores correm no namespace do próprio utilizador, sem permissões elevadas.
- Usa **user namespaces** para mapear o UID 0 (root) do contentor para um UID sem privilégios no anfitrião.
- Um contentor comprometido não consegue obter acesso root ao anfitrião.

```
# Sem sudo necessário
```

```
$ podman run --rm -it alpine sh
```

```
# Verificar: dentro do contentor é "root",  
# mas no anfitrião é o seu utilizador normal  
$ podman top -l user huser
```

Esta é uma **vantagem de segurança** significativa em ambientes de produção e partilhados.



# Podman: Compatibilidade de CLI

A interface de linha de comandos do Podman é **intencionalmente idêntica** à do Docker. Pode usar os mesmos comandos que já conhece:

```
$ podman pull nginx:alpine
$ podman run -d --name web -p 8080:80 nginx:alpine
$ podman ps
$ podman logs web
$ podman stop web
$ podman rm web
$ podman build -t myapp .
$ podman images
```

Em muitos sistemas, pode simplesmente criar um alias:

```
$ alias docker=podman
```

# Podman: Pods

O Podman introduz o conceito de **pods**, inspirado pelo Kubernetes.

Um pod é um grupo de contentores que:

- Partilham o mesmo **namespace de rede** (mesmo endereço IP, mesmo localhost).
- Partilham o mesmo **namespace IPC**.
- Podem comunicar via localhost sem mapeamento de portas.

```
# Criar um pod com mapeamento de portas
```

```
$ podman pod create --name webapp -p 8080:80
```

```
# Adicionar contentores ao pod
```

```
$ podman run -d --pod webapp --name web nginx:alpin
```

```
$ podman run -d --pod webapp --name api my-api-imag
```

```
# Listar pods
```

Para aplicações multi-contentor, o Podman suporta ficheiros Compose através de duas abordagens:

## 1. **podman compose (integrado, Podman 4.7+):**

```
$ podman compose up -d  
$ podman compose down
```

## 2. **podman-compose (ferramenta Python de terceiros):**

```
$ pip install podman-compose  
$ podman-compose up -d
```

Ambos leem ficheiros `compose.yml` / `docker-compose.yml` padrão. A maioria dos ficheiros Compose funciona sem modificação.

## Podman: Integração com Systemd

O Podman pode gerar **ficheiros de unidade systemd** para gerir contentores como serviços do sistema. Isto significa que os contentores arrancam no boot e são monitorizados pelo sistema init.

```
# Gerar uma unidade systemd para um contentor em ex  
$ podman generate systemd --new --name web > web.se
```

```
# Instalar e ativar o serviço (rootless)  
$ mkdir -p ~/.config/systemd/user/  
$ mv web.service ~/.config/systemd/user/  
$ systemctl --user daemon-reload  
$ systemctl --user enable --now web.service
```

Isto substitui a política restart: always do Docker por um gestor de serviços nativo e adequado do SO.

# Podman: Geração de YAML Kubernetes

O Podman também pode gerar e consumir **YAML Kubernetes** diretamente, fazendo a ponte entre desenvolvimento local e implementação em produção:

```
# Gerar YAML Kubernetes a partir de um pod em execu
$ podman generate kube webapp > webapp.yml
```

```
# Implementar a partir de YAML Kubernetes localment
$ podman play kube webapp.yml
```

```
# Remover
$ podman play kube --down webapp.yml
```

Isto torna a transição do desenvolvimento local com Podman para um cluster Kubernetes muito mais suave.

# Docker vs. Podman: Comparação

| Característica         | Docker                       | Podman                               |
|------------------------|------------------------------|--------------------------------------|
| <b>Arquitetura</b>     | Cliente-servidor (daemon)    | Sem daemon (fork/exec)               |
| <b>Root necessário</b> | Sim (daemon corre como root) | Não (rootless por defeito)           |
| <b>CLI</b>             | <code>docker ...</code>      | <code>podman ...</code> (compatível) |
| <b>Compose</b>         | <code>docker compose</code>  | <code>podman compose</code>          |
| <b>Pods</b>            | Não suportado                | Suportado (estilo Kubernetes)        |
| <b>Systemd</b>         | Não integrado                | Integração nativa                    |
| <b>YAML Kubernetes</b> | Não suportado                | Gerar e reproduzir                   |
| <b>GUI Desktop</b>     | Docker Desktop               | Podman Desktop                       |
| <b>Compatível OCI</b>  | Sim                          | Sim                                  |
| <b>Defeito no RHEL</b> | Não                          | Sim                                  |

# Quando Usar Docker vs. Podman

## Escolha Docker quando:

- Está a aprender contentores pela primeira vez (comunidade maior, mais tutoriais).
- A sua equipa ou pipeline CI/CD já usa Docker.
- Precisa das funcionalidades do Docker Desktop (GUI, cluster Kubernetes, Extensões).

## Escolha Podman quando:

- A segurança é uma prioridade (rootless, sem daemon).
- Está em RHEL, Fedora ou CentOS Stream (pré-instalado).
- Está a desenvolver para Kubernetes (pods, geração de YAML).
- Precisa de integração com systemd para serviços de produção.
- A política da sua organização proíbe correr daemons como root.

Ambas as ferramentas produzem imagens compatíveis com

# Conclusão

---



- Os contentores resolvem o problema do “na minha máquina funciona”, empacotando uma aplicação com as suas dependências numa unidade **portátil**.
- Alcançam isolamento e gestão de recursos através de funcionalidades do kernel Linux: **namespaces** e **cgroups**.
- O **Docker** popularizou os contentores com um CLI amigável, Dockerfile e Docker Hub.
- O **Docker Compose** permite a gestão declarativa de aplicações multi-serviço.
- O **Podman** é uma alternativa moderna, sem daemon e sem root, compatível com o CLI do Docker, que acrescenta suporte para pods, integração com systemd e geração de YAML Kubernetes.
- A **norma OCI** garante que imagens e runtimes são interoperáveis entre ferramentas.

- **Documentação Oficial do Docker:** A referência definitiva para o CLI Docker, Dockerfile e Compose.
  - <https://docs.docker.com/>
- **Documentação Oficial do Podman:** Guias de início, tutoriais e referência para o Podman.
  - <https://podman.io/docs>
- **Folha de Consulta do Docker (Collabnix):** Uma folha de consulta abrangente com exemplos detalhados.
  - <https://dockerlabs.collabnix.com/docker/cheatsheet/>

- **Como Otimizar Imagens Docker (GeeksforGeeks):**  
Técnicas como builds multi-estágio para tornar imagens mais pequenas e seguras.
  - <https://www.geeksforgeeks.org/devops/how-to-optimize-docker-image/>
- **Podman vs Docker (Red Hat):** Uma comparação oficial dos criadores do Podman.
  - <https://www.redhat.com/en/topics/containers/what-is-podman>
- **LinuxServer.io:** Imagens de contentores de alta qualidade mantidas pela comunidade para aplicações auto-hospedadas populares.
  - <https://www.linuxserver.io/>